SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	**K. HEMASREE**	**Reg. No.:**	**192411156**
Section / Batch:	**SLOT C**	**Date:**	**28/07/26**

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT

Q23. A hospital system stores emergency patient data unsorted for quick updates. Doctors need to locate records rapidly.

a) Explain how Linear Search helps in this scenario.

b) Analyze its performance in different cases.

c) Justify why sorting may not always be preferred.

Code of Conduct

I _KEERTHIPATI HEMASREE(192411156)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex real world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision Making Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				

Total Marks (100):	

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q21. Financial System – Execution-Based Analysis

Problem Statement

A financial system continuously receives stock price updates and needs to sort them frequently for trend analysis. Compare **Selection Sort** and **Bubble Sort**, analyze their computational cost, and recommend the most suitable algorithm.

Given Input

Stock Prices:
[520, 510, 535, 500, 545]

Number of elements (**n = 5**)

a) Comparison of Selection Sort and Bubble Sort

Criteria	Selection Sort	Bubble Sort
Method	Selects the minimum element and places it in the correct position	Compares adjacent elements and swaps if necessary
Comparisons	Fixed	Depends on data order
Swaps	Minimum	Can be many

Best Case	$O(n^2)$	O(n) (Optimized)
Average Case	$O(n^2)$	$O(n^2)$
Worst Case	$O(n^2)$	$O(n^2)$
Adaptive	No	Yes
Suitable for Frequently Updated Data	No	Yes

b) Execution of Selection Sort

Initial Array

[520, 510, 535, 500, 545]

Pass 1

Find the smallest element.

Comparisons:

520 vs 510
510 vs 535
510 vs 500
500 vs 545

Minimum = **500**

Swap with first element.

[500, 510, 535, 520, 545]

Comparisons = **4**

Swaps = **1**

Pass 2

Remaining array

[510, 535, 520, 545]

Comparisons

510 vs 535
510 vs 520
510 vs 545

Minimum already in position.

[500,510,535,520,545]

Comparisons = **3**

Swaps = **0**

Pass 3

Remaining array

[535,520,545]

Comparisons

535 vs 520
520 vs 545

Minimum = 520

Swap

[500,510,520,535,545]

Comparisons = **2**

Swaps = **1**

Pass 4

Remaining

535 545

Comparison

535 vs 545

Already sorted.

Comparisons = **1**

Swaps = **0**

Selection Sort Execution Summary

Pass	Comparisons	Swaps	Array
1	4	1	500 510 535 520 545
2	3	0	500 510 535 520 545
3	2	1	500 510 520 535 545
4	1	0	500 510 520 535 545

Total Comparisons = 10

Total Swaps = 2

Execution of Bubble Sort

Initial Array

[520,510,535,500,545]

Pass 1

Compare

520 & 510 → Swap

510 520 535 500 545

Compare

520 & 535 → No Swap

Compare

535 & 500 → Swap

510 520 500 535 545

Compare

535 & 545 → No Swap

Result

510 520 500 535 545

Comparisons = **4**

Swaps = **2**

Pass 2

Compare

510 & 520 → No Swap

Compare

520 & 500 → Swap

510 500 520 535 545

Compare

520 & 535 → No Swap

Result

510 500 520 535 545

Comparisons = **3**

Swaps = **1**

Pass 3

Compare

510 & 500 → Swap

500 510 520 535 545

Compare

510 & 520 → No Swap

Result

500 510 520 535 545

Comparisons = **2**

Swaps = **1**

Pass 4

Compare

500 & 510

No Swap

Optimized Bubble Sort detects **no swaps**.

Algorithm terminates.

Comparisons = **1**

Swaps = **0**

Bubble Sort Execution Summary

Pass	Comparisons	Swaps	Array
1	4	2	510 520 500 535 545
2	3	1	510 500 520 535 545
3	2	1	500 510 520 535 545
4	1	0	500 510 520 535 545

Total Comparisons = 10

Total Swaps = 4

Computational Cost

For **n elements**

Selection Sort

Comparisons

$$
\frac{n(n-1)}{2}
$$

For **50 stock prices**

$$
\frac{50\times49}{2}=1225
$$

Swaps

$$
n-1=49
$$

Bubble Sort

Worst-case comparisons

$$
\frac{50\times49}{2}=1225
$$

Worst-case swaps

$$
1225
$$

Best-case comparisons (Optimized)

[
n-1=49
]

Best-case swaps

[
0
]

Complexity Analysis

Algorithm	Best	Average	Worst	Comparisons (n=50)
Selection Sort	O(n²)	O(n²)	O(n²)	1225
Bubble Sort	O(n)	O(n²)	O(n²)	49 to 1225

c) Recommendation

The stock prices in a financial system change continuously, meaning that after each update the list is usually **almost sorted** rather than completely random.

Selection Sort

- Always performs **1225 comparisons** for 50 stock prices.
- Does not take advantage of partially sorted data.
- Performs fewer swaps but wastes time in repeated comparisons.

Bubble Sort (Optimized)

- Detects if no swaps occur during a pass.
- Stops immediately when the list becomes sorted.
- Requires only **49 comparisons** in the best case.
- More efficient for continuously updated stock prices.

Final Recommendation

Optimized Bubble Sort is recommended because it adapts to nearly sorted data, reduces unnecessary comparisons, and provides faster execution when stock prices are updated frequently. Although both algorithms have a worst-case complexity of **$O(n^2)$**, Bubble Sort offers significantly better performance for this real-world scenario.